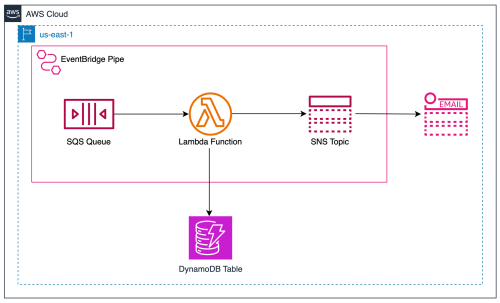
Getting Started with Serverless Event-Driven Architectures on AWS

Amazon EventBridge is an event management service. It allows us to connect components of a loosely coupled application through events, sparing developers the hassle of writing code to trigger or invoke components. EventBridge is effectively used to route, filter, and process events among AWS services, SaaS applications, and custom applications. EventBridge Pipes supports point-to-point integration by connecting a source and a target. Pipes allow us to filter and enrich the events sent to the destination.

In this Cloud Lab, you'll first create an SNS topic, an SQS queue, a Lambda function, and a DynamoDB table. You'll then create an Eventbridge Pipe, which will use the SQS topic as the source and filter the messages it receives. It will then invoke the Lambda function to enrich the messages it receives by fetching data from the DynamoDB table. Once the data is enriched, our message will be published on the SNS topic, which will then be forwarded to an email subscriber.

After completing this Cloud Lab, you can use EventBridge Pipes to create a serverless event-driven architecture. The following is the high-level architecture you will create in this Cloud Lab:



Getting Started

Amazon EventBridge allows us to easily build loosely coupled applications that support event-driven architecture. EventBridge provides us with an event bus that routes events from a source to a target using EventBridge rules. Through this service, we can enable our development teams to act independently by working on different components of an application, decreasing development time, and simplifying the process of adding or updating application components. EventBridge also provides us with pipes, which are used to connect a source and a target and support point-to-point integration. Pipes allow us to filter and enrich the events sent to a destination. In event filtering, only a subset of events are sent to the destination based on the defined criteria. Whereas, in event enrichement, we can add additional data to an event sent by the source before delivering it to its destination. For example, we can add an object's metadata to an event generated when an object is uploaded in an S3 bucket using a Lambda function.

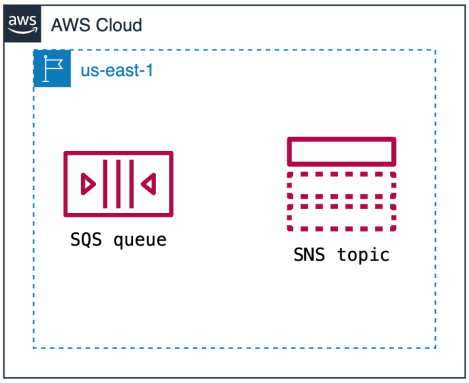
In this lab, we'll create an EventBridge pipe that receives messages from an SQS queue and delivers them to an SNS topic after filtering and enriching them.

Create an SQS Queue and an SNS Topic

Amazon Simple Queue Service (SQS) is a serverless service that allows us to integrate systems and send any volume of messages between them using a queue. We can create two types of queues using Amazon SQS:

- **Standard:** In standard queues, messages are sent to the queue at least once and can be unordered. We get nearly unlimited throughput, which means that standard queues support an almost unlimited number of transactions per second.
- **FIFO (first in, first out):** In FIFO queues, messages are sent exactly once to the consumer, and their order is maintained.

In this task we'll create a standard queue that will be used as the source in EventBridge Pipe. We'll also create an SNS topic, which will used as a target in our EventBridge Pipe. The messages sent to this queue will be filtered and enriched and sent to an SNS topic. After the completion of this task, the provisioned infrastructure should look like the one shown in the figure below:



Create an SQS queue

Follow the steps given below to create a queue using Amazon SQS:

- Search for “SQS” in the search bar and select “Simple Queue Service” from the results.
- Click the “Create queue” button to create a new queue.
- Select “Standard” as the “Type” of the queue and set its name to `clab-queue`.
- Leave all other settings to default and click the “Create queue” button.

Once our SQS queue is created, copy your queue’s URL, available under the “Details” section of the queue on the “Amazon SQS > Queues > clab-queue” page.

Create an SNS topic

Now, create an SNS topic that will receive filtered and enriched messages through an EventBridge Pipe and forward it to an email address we provide.
SNS supports two types of topics—standard and FIFO (first-in, first-out). Standard topics are used when the order of the messages being sent isn’t crucial. Message duplication is also possible in standard topics. On the other hand, FIFO topics are used when we want to ensure that messages are sent in the correct order and no duplicates are created.
Follow the steps given below to create an SNS topic:

- In the search bar, search “SNS” and select the “Simple Notification System” service.
- On the SNS dashboard, select the “Topics” tab from the left navigation bar and then click the “Create topic” button.
- On the “Create topic” page, select “Standard” as the topic type in the “Details” section, and set the name of the topic to `clab-topic`, and then click the “Create topic” button.

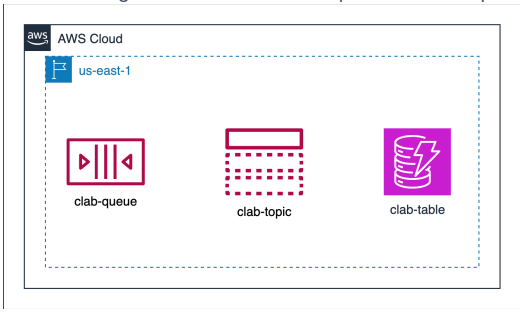
Now, add an email subscriber to this topic:

- On the SNS dashboard, select “Subscriptions” from the sidebar and click “Create subscription.”
- Select the ARN of the topic we created in “Topic ARN” from the drop-down list available.
- Set “Protocol” to “Email.”
- Provide a valid email address in “Endpoint.”
- Click “Create subscription.”

To start receiving messages published on our SNS topic, we need to confirm our email address. To do this, click the confirmation link sent to your email address.

Create a DynamoDB Table

Amazon DynamoDB is a NoSQL database service provided by Amazon. It supports the key-value and document models and provides various features, such as security, data backups, caching, and others. We can add data to the table through the console or through an SDK such as Boto3.
In this lab, we’ll create a DynamoDB table containing details about different fruits, such as their name, color, quantity, etc. This table will then be used to enrich messages received in the SQS queue. After completing this task, the provisioned infrastructure should be similar to the one shown below.



Create a table

Follow the steps given below to create a DynamoDB table.

- Search for “DynamoDB” in the search bar and select “DynamoDB” from the results.
- On the DynamoDB dashboard, click the “Create table” button.
- Set `clab-table` as the name of the table in the “Table name” section.
- Set the “Partition key” as `ItemID` and select “String” as the data type. This is the primary key of our table.
- Click the “Create table” button.

Add items to the table

Now that we have created a table, let’s add some items to it by executing the code given below. Wait for the status of the table to become “Active” before clicking the “Run” button below.
If you have your AWS account Access key and Secret then you can execute the below code to add items to the DynamoDB table. Or the data items can be

added manually on the DynamoDB table manually, follow the steps as below:

1. When the DynamoDB table is created, click on Tables from the dashboard and open the `clab-table`
2. Click on Actions and select Create Item
3. Type 1 as the Value for ItemID
4. Then click on Add new attribute and select String from the drop-down
5. Type Name as the attribute name and Banana as the Value
6. Then click on Add new attribute and select String from the drop-down
7. Type Type as the attribute name and Fruits as the Value
8. Then click on Add new attribute and select String from the drop-down
9. Type Color as the attribute name and Yellow as the Value
10. Then click on Add new attribute and select Number from the drop-down
11. Type Quantity as the attribute name and 50 as the Value
12. Repeat the above steps for the following data items

```
ItemID: 2
Name: Strawberry
Type: Fruits
Color: Red
Quantity: 30

ItemID: 3
Name: Grapes
Type: Fruits
Color: Green
Quantity: 40

ItemID: 4
Name: Potato
Type: Vegetables
Color: Brown
Quantity: 70
```

```
import boto3
from botocore.exceptions import ClientError

region = 'us-east-1'
accessKeyId = 'AKIAXHSXDSZXCZLIGRWN'
secretAccessKey = '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'

# DynamoDB resource
dynamodb = boto3.resource('dynamodb', region_name=region, aws_access_key_id=accessKeyId,
                          aws_secret_access_key=secretAccessKey)
table = dynamodb.Table('clab-table')

try:
    with table.batch_writer() as batch:
        batch.put_item(
            Item={
                'ItemID': '1',
                'Name': 'Grapes',
                'Type': 'Fruit',
                'Color': 'Green',
                'Quantity': 50
            }
        )
        batch.put_item(
            Item={
                'ItemID': '2',
                'Name': 'Strawberry',
                'Type': 'Fruit',
                'Color': 'Red',
                'Quantity': 30
            }
        )
        batch.put_item(
            Item={
                'ItemID': '3',
                'Name': 'Potato',
                'Type': 'Vegetable'
```

```
            'Color': 'Brown',
            'Quantity': 70
        }
    )
    print('Items added to the table successfully!')

except Exception as e:
    print(f"Error adding items to DynamoDB table: {e}")
```

View items in `DynamoDBTable`

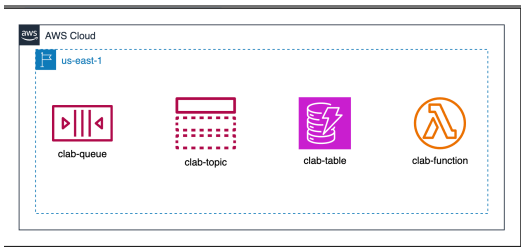
Now that we have created a table and added data, let's fetch the items from our table to confirm whether the items have been added. To do this, follow these steps:

- Navigate to the DynamoDB console.
- From the "Tables" page, click "clab-table" and then click the "Explore table items" button.

The items we added should be visible under the "Items returned" section.

Create a Lambda Function

AWS Lambda is a serverless computing service that runs our code in response to some event. We can invoke/trigger our Lambda functions through different AWS services. In this lab, it will be used to enrich SQS messages.
So far, we have created a topic, a queue, and a table; let's create the last component we need for our EventBridge Pipe. In this task, we'll create a Lambda function to receive messages from an SQS queue and then fetch items from a DynamoDB table to enrich this message. After the completion of this task, the provisioned infrastructure should look like the one shown in the figure below:



Create a Lambda function by following the steps given below:

- Search for "Lambda" in the search bar and select "Lambda" from the results.
- Click the "Create a function" button.
- Select the "Author from scratch" option and set `clab-function` as the "Function name."
- Select "Python 3.14" from the "Runtime" drop-down list.
- Click the "Additional settings" in the "Custom settings" section to expand it.
    - Click the "Toggle" button from the "Custom execution role" card in the "General" section. It will open the "Configure custom execution role" menu on the right side.
    - Select the "LambdaExecutionRole" role from the "Execution role" drop-down list and click the "Save" button.

Once the function is created, navigate to the "Code" tab, copy and paste the following code into the `lambda_function.py` file and click the "Deploy" button to save the changes made to the code:

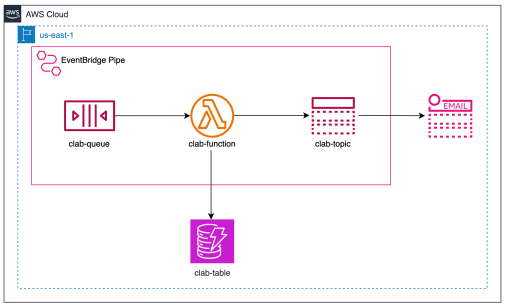
```
import json
import boto3

dynamodb = boto3.client('dynamodb')

def lambda_handler(event, context):
    print(f"Received EventBridge event: {json.dumps(event)}")

    # Iterate over each record in the event
    for record in event:
        sqs_message = json.loads(record['body'])
        print(f"Received SQS message: {json.dumps(sqs_message)}")

        # Fetch the key from the sqs message
        ItemID = sqs_message.get('ItemID')

        # Fetch data from DynamoDB table
        dynamodb_response = dynamodb.get_item(
            TableName='clab-table',
            Key={
```

```
            'itemID': {'S': itemID}
        }
    )

    # Process DynamoDB response
    enriched_data = dynamodb_response.get('Item', {})

    # Add enriched data to SQS message
    sqs_message['enriched_data'] = enriched_data

    # Print enriched message
    print(f"Enriched SQS message: {json.dumps(sqs_message)}")


# Return a response if needed
return {
    'statusCode': 200,
    'body': sqs_message
}
```

Now that we have created all the components required, we'll create an EventBridge Pipe and see how it works.

Create an EventBridge Pipe

Now that we have created all the components required to create an EventBridge Pipe, let's finally integrate them and see how EventBridge Pipes work. In this pipe, we'll use our SQS queue as the source, our Lambda function for the data enrichment, and our SNS topic as the target.
After the completion of this task, the provisioned infrastructure should look like the one shown in the figure below:



Create an IAM role `clab-eventbridge-role` and attach the below permissions:

```
{
   "Version":"2012-10-17",
   "Statement":[
      {
         "Effect":"Allow",
         "Action":[
            "sqs:ReceiveMessage",
            "sqs:DeleteMessage",
            "sqs:GetQueueAttributes",
            "lambda:InvokeFunction",
            "sns:Publish"
         ],
         "Resource":"*"
      }
   ]
}
```

Follow the steps given below to create an EventBridge Pipe:

- Search for "EventBridge" in the search bar and select "Amazon EventBridge" from the results.
- Select "Pipes" from the left navigation bar and then click the "Create pipe" button.
- Provide `clab-pipe` as the name of the pipe and press the "Enter" key.
- Open the "Pipe settings" tab and select "Use existing role" as the "Execution role."
- Select "clab-eventbridge-role" from the "Role name" drop-down. This role contains the permissions required by Amazon EventBridge to receive SQS messages, invoke Lambda functions, and publish messages on an SNS topic

- Now open the "Build pipe" tab. Here, we'll configure the source, filtering, enrichment, and target resources for our pipe.

Add SQS queue as the source

First, let's add an SQS queue as a source in our EventBridge Pipe by following the steps given below:

- In the "Source" tab, select "SQS" from the drop-down available under "Source" in the "Details" section.

- Select `clab-queue` from the drop-down given under “SQS queue” and then click the “Next” button.

Enable filtering in the pipe

Now, let’s enable filtering in our pipe so that we only receive messages where the type of item is `Fruit`. All other messages will be discarded once we enable this option. In the “Filtering - *optional,*” follow the steps given below to filter events received by the EventBridge Pipe:

- In the “Sample event - *optional*” section, select the “Enter my own” option and copy and paste the following sample event in the JSON editor. This is a sample of the type of events generated when a message is sent to the SQS queue. We have also defined the body of the message on **lines 4–7**.

```
{
 "messageId": "059f36b4-87a3-44ab-83d2-661975830a7d",
 "receiptHandle": "AQEBwJnKyrHigUMZj6rYigCgxlaS3SLy0a...",
 "body": {
  "Name": "Strawberry",
  "Type": "Fruit"
 },
 "attributes": {
  "ApproximateReceiveCount": "1",
  "SentTimestamp": "1545082649183",
  "SenderId": "AXXXIIIIZZZZZ222220O",
  "ApproximateFirstReceiveTimestamp": "1545082649185"
 },
 "messageAttributes": {},
 "md5OfBody": "e4e68fb7bd0e697a0ae8f1bb342846b3",
 "eventSource": "aws:sqs",
 "eventSourceARN": "arn:aws:sqs:us-east-1:*:clab-queue",
 "AWS region": "us-east-1"
}
```

- In the “Event pattern 1” section, select “Equals-ignore-case matching” from the drop-down menu. With this option, our pipe will only forward events where the messages include a field having a value we define, regardless of its case.
- Copy and paste the following event pattern in the JSON editor available. This event pattern matches any event that has a `Type` field that matches the string `fruit`, regardless of its case.

```
{
  "body": {
    "Type": [ { "equals-ignore-case": "Fruit" } ]
  }
}
```

After providing this event pattern, click the “Test pattern” button. You’ll see a pop-up saying, “Sample event matched the event pattern.” This means the sample event we provided matches the event pattern we have provided. You can play around here by providing different message bodies on **lines 4–7** of the sample pattern and clicking the “Test pattern” button.
Click the “Next” button to configure the enrichment settings for our pipe.

Enrich events in the pipe

In this section, we’ll connect the Lambda function we created earlier with our EventBridge Pipe to enrich the filtered events we receive from our source. Through this, the Lambda function will fetch data from the DynamoDB table and add it to the message received by the SQS queue. Follow the steps given below to configure the enrichment section for this pipe.

- In the “Enrichment - *optional*” section, select “AWS Lambda” from the drop-down menu given under “Service.”
- Select `clab-function` from the drop-down menu given under “Function” and then click the “Next” button.

Configure the target

In this last step, we must ensure that our pipe runs smoothly by setting up a target. Let’s add the SNS topic we created earlier as a target in this pipe by following the steps given below:

- In the “Target” section, select “SNS Topic” from the drop-down available under “Target service” in the “Details” section.
- Select `clab-topic` from the drop-down given under “Topic” and then click the “Create pipe” button.

Congratulations! We’ve created an EventBridge Pipe and configured its source, filtering, enrichment, and target resources. Now let’s test this pipe in the next task.

Test the EventBridge Pipe

Now that we have created an EventBridge Pipe and configured all the resources required to execute it successfully, let’s see the pipe in action. In this task, we won’t be configuring any new resources; we will just test out the existing architecture we have created.

Send messages to the SQS queue

We will use Boto3, which is an AWS SDK for Python, to send messages to our queue.

In the following code, we use the `send_message()` function available in Boto3 to send messages to the queue we created. Make sure to provide your queue’s

In the following code, we use the `send_message()` function available in Boto3 to send messages to the queue we created. Make sure to provide your queue's URL, available under the "Details" section of the queue, on **line 8** in the code below:

```
import boto3

# Create an SQS client
client = boto3.client('sqs',region_name='us-east-1',aws_access_key_id="AKIAXHSXDSZXCZLIGRWN",
      aws_secret_access_key= "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")

# URL of the queue
queue_url = 'https://sqs.us-east-1.amazonaws.com/497325151854/clab-queue'

# Send message to SQS queue
response_1 = client.send_message(
   QueueUrl=queue_url,
   MessageBody=(
      '{ "ItemID": "1", "Type": "Fruit"}'
   )
)

response_2 = client.send_message(
   QueueUrl=queue_url,
   MessageBody=(
      '{ "ItemID": "2", "Type": "Fruit"}'
   )
)

response_3 = client.send_message(
   QueueUrl=queue_url,
   MessageBody=(
      '{ "ItemID": "3", "Type": "Vegetable"}'
   )
)

print('Messages sent successfully')
```

Or send messages directly from the AWS Management console, follow the below steps:

1. Goto the SQS Dashboard, click on queues
2. Selected the clab-queue and click on Send and receive messages
3. In the message body type:

   ```
   "{ \"ItemID\": \"2\", \"Type\": \"Fruit\"}"
   ```

   ```
   '{ "ItemID": "2", "Type": "Fruit"}'
   ```

4. And click Send message

It should a success message on the Pop-Up and you should receive an email with the notification.

Clean Up

The final task we should perform is to remove all the resources we created. Follow the steps given under each task to delete the resources we created.

Delete the EventBridge Pipe

- Search for "EventBridge" in the search bar and open "Amazon EventBridge."
- Click "Pipes" from the sidebar.
- Click the pipe we created and click the "Delete" button.
- Type `delete` in the given text field of the delete pipe prompt and click the "Delete" button.

Remove the SQS queue

- Search for "SQS" in the search bar and open the "Simple Queue Service."
- Select the "Queues" tab from the sidebar.
- Select the queue `clab-queue`, and then click the "Delete" button.
- Enter the phrase `confirm` in the text box and click the "Delete" button.

Remove the SNS topic

- Search "SNS" in the search bar and open the "Simple Notification Service."
- Select the "Topics" tab from the sidebar.
- Select `clab-topic` and click the "Delete" button. Enter the phrase, `delete me`, and click the "Delete" button.

Delete the DynamoDB table

- Search for “DynamoDB” in the search bar and open “DynamoDB.”
- Click “Tables” from the sidebar.
- Select the table `clab-table` and click “Delete.”
- Enter the phrase `confirm` in the prompt and click “Delete.”

Delete the Lambda function

- Navigate to “Lambda > Functions” and select the function we created earlier.
- Click the “Actions” tab and select the “Delete” option from the drop-down.
- Type `delete` in the given text field of the delete function prompt, click the “Delete” button.